

Conversiono

AI Behavioral Intelligence for E-Commerce

Epic 2: ML Ensemble + SHAP

CONV-22 to CONV-27 | Sprint 2

*A course-style guide to ensemble ML, explainability,
and psychology-driven intervention systems*

Date: March 2026

Version: 1.0

Table of Contents

1. Prerequisites & What You'll Learn
2. Ensemble Models: Combining Multiple Classifiers
3. SHAP Explainability: Why Did the Model Decide That?
4. Psychology-Aware Interventions: From ML to Action
5. Real-Time Scoring Service: Putting It All Together
6. Model Evaluation & A/B Testing
7. Integration Testing: Validating the Pipeline
8. Architecture & Data Flow Diagrams
9. Exercises & Challenges

Chapter 1

Prerequisites & What You'll Learn

1.1 What Epic 1 Built (Recap)

Before diving into Epic 2, let's recap what was built in Epic 1 (Data Foundation & ML Baseline). Understanding this foundation is essential because every component in Epic 2 builds directly on it.

The Three Models

Epic 1 trained three independent XGBoost classifiers, each solving a different prediction problem using the same 8 behavioral features extracted from user browsing sessions:

1.

Conversion Predictor (predictor_v1.pkl) - Binary classifier: will this user purchase or abandon? Trained with SMOTE oversampling to handle the 4% conversion rate imbalance. Target AUC ≥ 0.70 .
2.

The 8 Behavioral Features

Feature	Type	What It Measures
hesitation_score	0-1	Pauses before clicks (decision paralysis)
price_dwell_time_s	seconds	Time on price elements (price sensitivit
rage_click_score	0-1	Rapid repeated clicks (frustration)
scroll_retreat_count	int	Scroll direction reversals (indecision)
exit_intent_count	int	Attempts to leave (abandonment signal)
checkout_hesitation_s	seconds	Idle time at checkout (commitment fear)
velocity_variance	float	Mouse speed variance (erratic behavior)
session_duration_s	seconds	Total session length

The Psychology Framework

Epic 1 also defined 8 psychological states in framework_v1.json, each mapping behavioral feature thresholds to evidence-based interventions. For example, 'Loss Aversion' triggers when checkout_hesitation ≥ 15 s AND price_dwell ≥ 10 s.

1.2 What You'll Learn in This Guide

By the end of this guide, you will understand:

1. How to combine multiple ML models into an ensemble that is more reliable than any single model alone

2.

Chapter 2

Ensemble Models: Combining Multiple Classifiers

2.1 Why Not Just Use One Model?

Imagine you're a doctor diagnosing a patient. Would you rely on a single test, or would you combine blood work, imaging, and symptoms? The same principle applies to ML. Each of our three models sees the user session from a different angle:

- The conversion predictor asks: 'Will they buy?'

A user might have high conversion probability (they've added items to cart) but also high friction (they're rage-clicking the checkout button). No single model captures this full picture. An ensemble does.

2.2 Our Ensemble Approach: Weighted Combination

There are two common approaches to ensembles: (1) train a meta-model that learns to combine base model outputs (stacking), or (2) use a transparent weighted average. We chose option 2 because:

- Interpretability: we can explain exactly why a score is what it is

The Session Score Formula

The ensemble produces a single 'session_score' between 0 and 1, representing overall abandonment risk. The formula is:

```
session_score = (  
    0.5 * (1 - conversion_prob)    # Higher conversion = lower risk  
    + 0.3 * friction_prob          # Higher friction = higher risk  
    + 0.2 * intent_risk            # 'exiting' = 0.9 risk, 'browsing' = 0.3  
)
```

Key Insight: Inverting the Conversion Signal

Notice we use $(1 - \text{conversion_prob})$. This is because the conversion model predicts likelihood of PURCHASE, but our ensemble measures RISK. High purchase probability = low risk. This inversion is easy to forget and causes subtle bugs if you get the direction wrong.

Why These Weights?

The default weights (0.5, 0.3, 0.2) reflect business priority: conversion prediction is the most direct signal, friction adds diagnostic value, and intent provides behavioral context. These weights are configurable via EnsembleConfig, making it easy to tune them per merchant or run ablation studies.

2.3 The Action Ladder

The session score maps to one of four actions, creating a graduated response system. This prevents over-intervention (annoying low-risk users) while ensuring high-risk sessions get immediate attention:

Score Range	Action	What Happens
< 0.30	monitor	No intervention. User is fine.
0.30 - 0.49	nudge	Soft engagement (subtle trust signals)
0.50 - 0.74	intervene	Active intervention (targeted help)
>= 0.75	urgent	Last-chance retention (exit discount)

2.4 Confidence: How Sure Are We?

Besides the score, the ensemble computes a 'confidence' metric that measures how much the three models agree. If all three models point in the same direction (all say risky OR all say safe), confidence is high. If they disagree (conversion says safe but friction says dangerous), confidence drops.

This is critical for the business: a high-risk score with low confidence might mean 'wait and gather more data' rather than 'intervene immediately'. Confidence is computed from:

- Each model's certainty (distance from the 0.5 decision boundary)

2.5 Implementation (CONV-22)

File: ml/src/ensemble/ensemble.py

```
from src.ensemble.ensemble import EnsembleModel

ensemble = EnsembleModel() # default weights
result = ensemble.score(
    conversion_prob=0.8,
    intent_label='browsing',
    intent_probs={'browsing': 0.7, 'comparing': 0.2, ...},
    friction_prob=0.2,
)
print(result.session_score)      # 0.22 (low risk)
print(result.recommended_action) # 'monitor'
print(result.confidence)         # 0.85 (high agreement)
```

Chapter 3

SHAP: Why Did the Model Decide That?

3.1 The Explainability Problem

XGBoost models are powerful but opaque. When the friction detector says 'this session has 87% friction probability', a merchant rightfully asks: 'Why? What specifically is causing friction?' Without an answer, the prediction is useless for action.

This is where SHAP comes in. SHAP is based on game theory (specifically Shapley values from cooperative game theory, 1953) and provides a mathematically rigorous way to attribute each feature's contribution to a specific prediction.

3.2 How SHAP Works (Intuition)

Imagine you're splitting a restaurant bill fairly among friends who ordered different items. Shapley values solve this exact problem: they compute each player's fair contribution by considering all possible orderings of players.

For ML, 'players' are features and the 'bill' is the prediction:

1.

Start with the base prediction (average model output)
2.

The Additivity Property

SHAP values have a beautiful property: they always sum to the difference between the prediction and the base value. If base = 0.3 and prediction = 0.87, the 8 SHAP values will sum to exactly 0.57. This makes explanations consistent and trustworthy.

3.3 Local vs Global Explanations

Local: Per-Session

A local explanation answers: 'For THIS specific session, which features drove the prediction?' For example:

```
Session #42 - Friction probability: 0.87
checkout_hesitation_s = 45.0  [SHAP: +0.32]  increases_risk
rage_click_score = 0.6       [SHAP: +0.21]  increases_risk
session_duration_s = 30.0    [SHAP: -0.08]  decreases_risk
```

This tells us: 'Session #42 has high friction mainly because of 45 seconds of checkout hesitation and a rage click score of 0.6.'

Global: Across All Sessions

A global explanation answers: 'Across all sessions, which features matter most?' This is computed as the mean absolute SHAP value across a dataset. It tells you which features the model relies on most heavily.

3.4 Risk Direction Logic

A subtle but critical detail: the meaning of a positive SHAP value depends on which model you're explaining:

Model	Positive SHAP Means	Risk Direction
Conversion	Higher purchase probability	DECREASES risk
Friction	Higher friction probability	INCREASES risk
Intent	Higher predicted-class probability	Depends on class

The UnifiedExplainer handles this automatically via a direction map, so the FeatureAttribution.direction field always tells you whether a feature 'increases_risk' or 'decreases_risk' regardless of which model generated the SHAP values.

3.5 Implementation (CONV-23)

Files: ml/src/explainability/shap_explainer.py + modifications to training scripts

```
from src.explainability.shap_explainer import UnifiedExplainer

explainer = UnifiedExplainer.from_artifacts(Path('ml/artifacts'))

# Local explanation for one session
local = explainer.explain_local(features, 'friction', top_k=3)
for attr in local.top_k:
    print(f'{attr.feature_name}: {attr.shap_value} ({attr.direction})')

# Global explanation across a dataset
glob = explainer.explain_global(X_test, 'conversion')
print(glob.feature_ranking) # ['checkout_hesitation_s', ...]
```

Multi-Class SHAP Gotcha

For the intent classifier (4 classes), SHAP returns values per class. Newer versions of SHAP return a 3D array (samples x features x classes) while older versions return a list of 2D arrays. Our UnifiedExplainer handles both formats automatically. When writing your own SHAP code, always check the shape!

Chapter 4

Psychology-Aware Interventions

4.1 Bridging ML and Business Action

This is where Conversiono's true value proposition lives. Many ML systems stop at prediction: 'this user will abandon with 73% probability.' But a merchant can't act on that number. They need to know WHAT to do.

The Intervention Engine bridges this gap by connecting SHAP explanations to the psychology framework. It answers: 'Given that checkout_hesitation and price_dwell are driving this session's friction, the user is likely experiencing Loss Aversion - show them a money-back guarantee.'

4.2 Two-Layer Matching Logic

The engine uses two complementary layers to determine interventions:

Layer 1: Rule-Based Trigger Matching

Each psychological state in the framework has trigger conditions. For example, Loss Aversion triggers when `checkout_hesitation_s >= 15 AND price_dwell_time_s >= 10`. The engine evaluates these conditions against the session's actual features.

The `trigger_logic` field controls how conditions combine: 'ALL' means every condition must be true (AND logic). 'ANY' means at least one condition must be true (OR logic). UI Frustration uses ANY - either rage clicks OR erratic mouse movement is enough to trigger it.

Layer 2: SHAP-Enhanced Confidence

Here's the clever part: just because a feature crosses a threshold doesn't mean the ML model actually relied on it. A session might have high `checkout_hesitation` (triggering Loss Aversion rules) but the friction model might have scored it as high-friction mainly because of `rage_clicks`, not `checkout_hesitation`.

SHAP-enhanced confidence solves this. When SHAP attributions are provided, the engine checks: 'Did the trigger features also have high SHAP values?' If yes, confidence gets a boost (up to +0.2). This means the ML model AGREES with the psychology framework's assessment, making the intervention more reliable.

4.3 Priority Scoring

When multiple psychological states are triggered (common for high-friction sessions), the engine must decide which interventions to show. Showing 5 different popups would be worse than showing none. The priority formula is:

```
priority_score = severity_weight * confidence * expected_lift
```

This naturally surfaces the most impactful interventions. A high-severity state (Abandonment Intent: 0.95) with high confidence and a high-lift intervention (exit discount: 22% expected lift) will always rank above a lower-severity state with a modest intervention.

4.4 The 8 Psychological States

State	Severity	Trigger Logic	Key Triggers
Abandonment Intent	0.95	ANY	exit_intent >= 2 OR duration <= 120
UI Frustration	0.90	ANY	rage_click >= 0.3 OR velocity >= 100k
Loss Aversion	0.85	ALL	checkout_hes >= 15 AND price_dwell >= 10
Trust Gap	0.80	ALL	price_dwell >= 8 AND checkout_hes >= 20
Price Shock	0.75	ALL	price_dwell >= 15 AND checkout_hes <= 5
Approach-Avoidance	0.72	ALL	hesitation >= 0.3 AND exit >= 1 AND retr
Decision Fatigue	0.70	ALL	duration >= 300 AND retreat >= 5
Decision Uncertainty	0.65	ALL	hesitation >= 0.4 AND retreat >= 3

4.5 Implementation (CONV-24)

File: ml/src/intervention/engine.py

```
from src.intervention.engine import InterventionEngine

engine = InterventionEngine()
features = {'checkout_hesitation_s': 25.0, 'price_dwell_time_s': 12.0, ...}

# Match psychological states
states = engine.match_states(features, shap_attributions)
for s in states:
    print(f'{s.state_label}: confidence={s.confidence}')

# Get ranked interventions
recs = engine.recommend(features, shap_attributions, max_interventions=3)
for r in recs:
    print(f'{r.intervention_id}: {r.message} (lift: {r.expected_lift})')
```

Chapter 5

Real-Time Scoring Service

5.1 The Orchestration Challenge

We now have 4 separate components: 3 ML models, an ensemble combiner, a SHAP explainer, and an intervention engine. The scoring service's job is to orchestrate them into a single, fast, easy-to-use API.

For each session, the service executes this pipeline:

1. Convert feature dict to ordered numpy array (matching FEATURE_COLS)

2.

5.2 Design Decisions

Singleton Pattern

Loading 3 XGBoost models + 3 SHAP explainers from disk takes ~200ms. We can't afford this per request. The ScoringService uses a singleton pattern (`get_instance()`) so models are loaded once at startup and reused for every subsequent request.

explain=True/False Toggle

SHAP computation adds ~50-100ms per session. For real-time scoring (single session, user-facing), we want full explanations. For batch scoring (model evaluation, data pipeline), we skip SHAP for speed. The `explain` parameter controls this.

Feature Dict Reordering

XGBoost is sensitive to feature order - the features must be in the same order as during training. Callers might pass features in any order. The service internally reorders them to match `FEATURE_COLS`, preventing subtle prediction errors.

5.3 The ScoringResult Object

Every call to `score_session` returns a `ScoringResult` containing:

Field	Type	Description
session_score	float	Unified abandonment risk 0-1
recommended_action	str	monitor/nudge/intervene/urgent
confidence	float	Model agreement confidence 0-1
conversion_prob	float	Purchase probability 0-1
intent_label	str	Predicted intent class
intent_probs	dict	Per-class probabilities
friction_prob	float	Friction probability 0-1
top_shap_features	dict	Top-3 SHAP per model
interventions	list	Ranked InterventionRecommendations
psychological_states	list	Matched PsychologicalStateMatch
latency_ms	float	End-to-end scoring time in ms

5.4 Implementation (CONV-25)

File: ml/src/scoring/service.py

```

from src.scoring.service import ScoringService

# Load models once
service = ScoringService.get_instance(
    artifacts_dir='ml/artifacts'
)

# Score a session
result = service.score_session({
    'hesitation_score': 0.4,
    'price_dwell_time_s': 12.0,
    'rage_click_score': 0.1,
    'scroll_retreat_count': 3,
    'exit_intent_count': 1,
    'checkout_hesitation_s': 18.0,
    'velocity_variance': 50000.0,
    'session_duration_s': 200.0,
})

print(f'Risk: {result.session_score} -> {result.recommended_action}')
print(f'Latency: {result.latency_ms}ms')
for iv in result.interventions:
    print(f' -> {iv.intervention_id}: {iv.message}')
```

Chapter 6

Model Evaluation & A/B Testing

6.1 Why Standard Accuracy Isn't Enough

With a 4% conversion rate, a model that ALWAYS predicts 'abandon' would have 96% accuracy. Clearly, accuracy alone is misleading for imbalanced datasets. That's why we track multiple metrics:

- AUC-ROC: How well can the model distinguish classes at all thresholds?

6.2 Comparing Model Versions

When you train a new model version (e.g., predictor_v2 with different hyperparameters), you need to know if it's ACTUALLY better, not just luckier on this particular test set. The ModelComparator uses McNemar's test - a statistical test specifically designed for comparing paired classifiers.

McNemar's Test (Intuition)

McNemar's test only looks at disagreements between the two models. If model A gets sample #42 right but model B gets it wrong (or vice versa), those are the informative cases. If both agree, it doesn't tell us which is better. The test asks: 'Are the disagreements symmetric, or does one model consistently win?'

6.3 A/B Testing Interventions

The ABTestAnalyzer computes statistical significance for intervention experiments. When you run an A/B test (control: no intervention vs treatment: show money-back guarantee), it tells you:

- Is the conversion rate difference statistically significant? (two-proportion z-test)

Sample Size Matters

A common mistake is stopping A/B tests too early. With $n=50$ per group, you can't reliably detect effects smaller than ~15%. For a typical 4% baseline with 2% MDE (minimum detectable effect), you need ~2,000 sessions per variant. The `required_sample_size()` method computes this for you.

6.4 Implementation (CONV-26)

Files: ml/src/evaluation/metrics.py and ml/src/evaluation/comparison.py

```
from src.evaluation.metrics import evaluate_binary
from src.evaluation.comparison import ABTestAnalyzer

# Evaluate a model
metrics = evaluate_binary(y_true, y_pred, y_proba)
print(f'AUC: {metrics.auc_roc}, F1: {metrics.f1}')

# Analyze an A/B test
ab = ABTestAnalyzer()
result = ab.analyze(
    control_conversions=100, control_total=1000,
    treatment_conversions=150, treatment_total=1000,
)
print(f'Lift: {result.lift:.1%}, p={result.p_value:.4f}')
# Lift: 50.0%, p=0.0007 -> statistically significant!
```

Chapter 7

Integration Testing: Validating the Pipeline

7.1 Why Integration Tests Matter

Unit tests verify each component in isolation: the ensemble math is correct, the SHAP explainer returns the right shape, the intervention engine matches the right states. But components that work individually can fail when combined. Integration tests catch these failures.

Common integration failures we test for:

- Feature column mismatch: model trained on features in order A, service passes them in order B

7.2 Scenario-Based Tests

The most valuable integration tests are scenario-based: they simulate a realistic user session and verify the entire pipeline produces sensible results. Here are the key scenarios we test:

Scenario: High Friction User

Features: rage_click=0.6, velocity_variance=200000. Expected: UI Frustration state triggered, 'live_help' or 'report_issue' intervention recommended.

Scenario: Loss Aversion Shopper

Features: checkout_hesitation=25s, price_dwell=15s. Expected: Loss Aversion state triggered, 'money_back_guarantee' recommended.

Scenario: Happy Browser

Features: all low signals. Expected: no states triggered, action='monitor', no interventions.

7.3 Cross-Component Contract Tests

These tests verify that components agree on shared contracts:

- All 3 model metadata files reference the exact same FEATURE_COLS

7.4 Test Results

All 145 tests pass (90 from Epic 2 + 55 from Epic 1):

Test File	Tests	Status
test_ensemble.py	21	PASS
test_shap_explainer.py	14	PASS
test_intervention_engine.py	14	PASS
test_scoring_service.py	15	PASS
test_evaluation.py	14	PASS
test_integration_pipeline.py	12	PASS
Epic 1 tests (unchanged)	55	PASS

Chapter 8

Architecture & Data Flow

8.1 Module Dependency Graph

The Epic 2 modules are organized in a clean dependency hierarchy:

```
ml/src/  
  data/loader.py          <- FEATURE_COLS definition (shared constant)  
  features/extraction.py  <- Raw events -> 8 features  
  training/               <- Train & save models (+ SHAP explainers)  
    train_xgboost.py  
    train_intent.py  
    train_friction.py  
  ensemble/ensemble.py   <- Weighted model combination (CONV-22)  
  explainability/        <- SHAP wrapper for all models (CONV-23)  
    shap_explainer.py  
  intervention/engine.py  <- Psychology framework matching (CONV-24)  
  evaluation/             <- Metrics & comparison (CONV-26)  
    metrics.py  
    comparison.py  
  scoring/service.py      <- Orchestrator (CONV-25) depends on all above
```

8.2 Data Flow: Session Scoring

When a session is scored, data flows through this pipeline:

Chapter 9

Exercises & Challenges

These exercises will deepen your understanding of the concepts covered in this guide. They range from beginner (modify existing code) to advanced (design new features).

9.1 Beginner Exercises

1. Weight Ablation: Modify EnsembleConfig to set intent_weight=0 and re-run the ensemble tests. What changes? Why might you want to disable the intent signal for some merchants?

2.

9.2 Intermediate Exercises

4. Custom Psychology State: Add a 9th psychological state to framework_v1.json called 'Social Comparison' that triggers when session_duration >= 400 AND scroll_retreat >= 8. Create appropriate interventions and verify the engine detects it.

5.

9.3 Advanced Challenges

7. Stacking Ensemble: Replace the weighted combination with a logistic regression meta-model that takes the 3 model outputs as features. Does it outperform the weighted approach? (Hint: you'll need a held-out fold for generating meta-features.)

8.

Summary

Epic 2 transformed three independent ML models into an integrated intelligence system. The ensemble combines their predictions into a unified risk score. SHAP explains WHY each prediction was made. The intervention engine maps those explanations to psychology-backed recommendations. And the scoring service orchestrates it all in real-time, ready for the backend API to call.

This architecture is intentionally modular: each component can be tested, upgraded, or replaced independently. The psychology framework is a JSON file, not hardcoded logic. The ensemble weights are configurable. The SHAP layer handles multiple model formats transparently.

Next up: Epic 3 (JavaScript SDK) will capture browser events, Epic 4 (Backend API) will expose the scoring service via REST endpoints, and Epic 5 (Dashboard) will visualize everything for merchants.

Statistics

- 6 stories implemented (CONV-22 to CONV-27)
- 5 new modules: ensemble, explainability, intervention, scoring, evaluation
- 2 existing training scripts enhanced with SHAP
- 90 new tests (145 total with Epic 1)
- 0 regressions